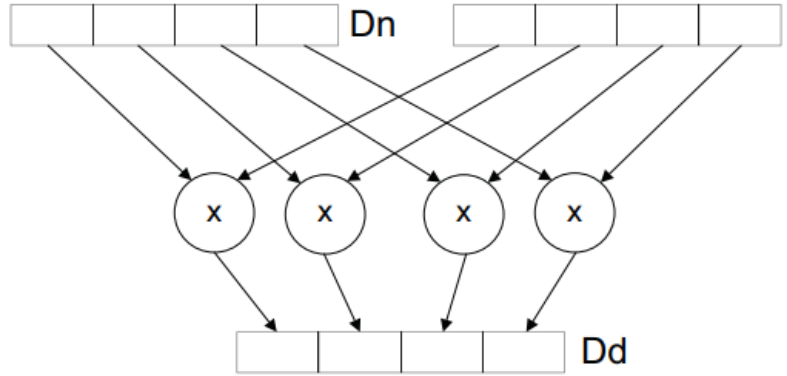
tp-link

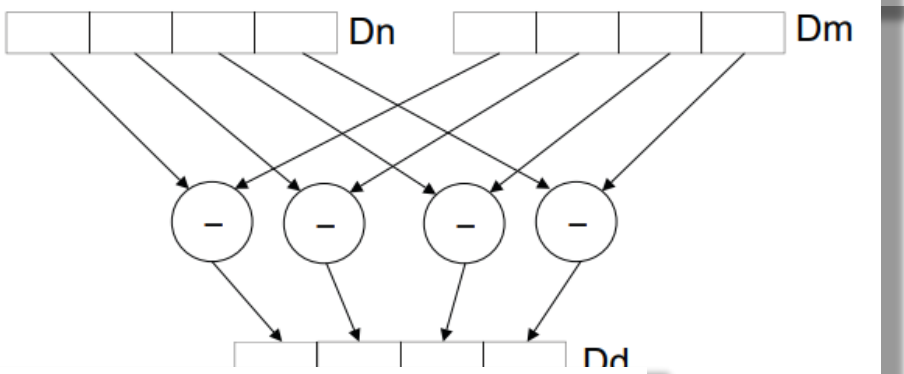
音频算法 NEON 优化

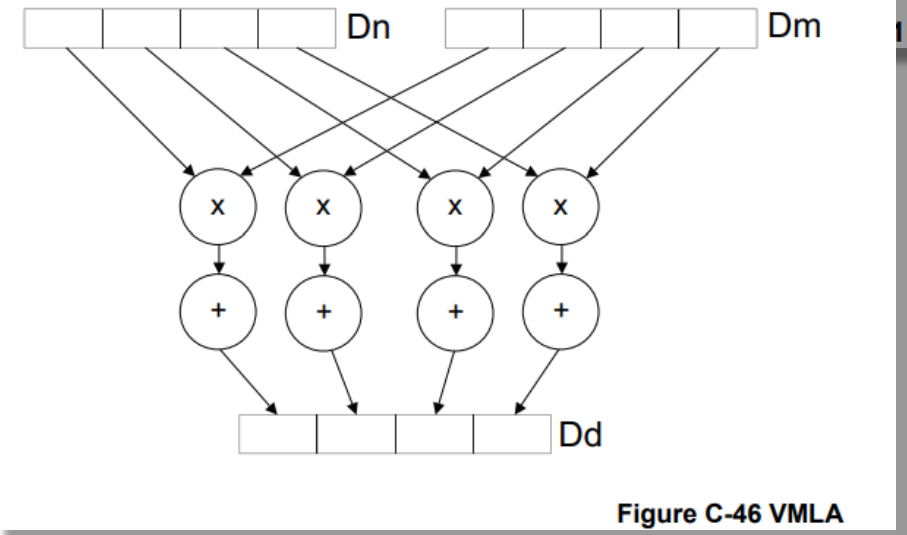
新人课题中期汇报

汇报人：刘元昊









45

VSUB operation

Figure C-46 VMLA

目录

课题背景简介

课题中期进度

创新点

后续工作方向

小结

tp-link

© 2020 TP-Link, Inc. Confidential information. Not for distribution or reproduction.

课题背景简介

背景

当前音频算法在算法原理上进行过多轮优化，原理上已逼近最优。考虑 NEON 优化，进一步**降低 CPU 消耗**。降低 CPU 消耗有助于应对芯片降本，CPU 余量可用于新增其他音频功能。

概念

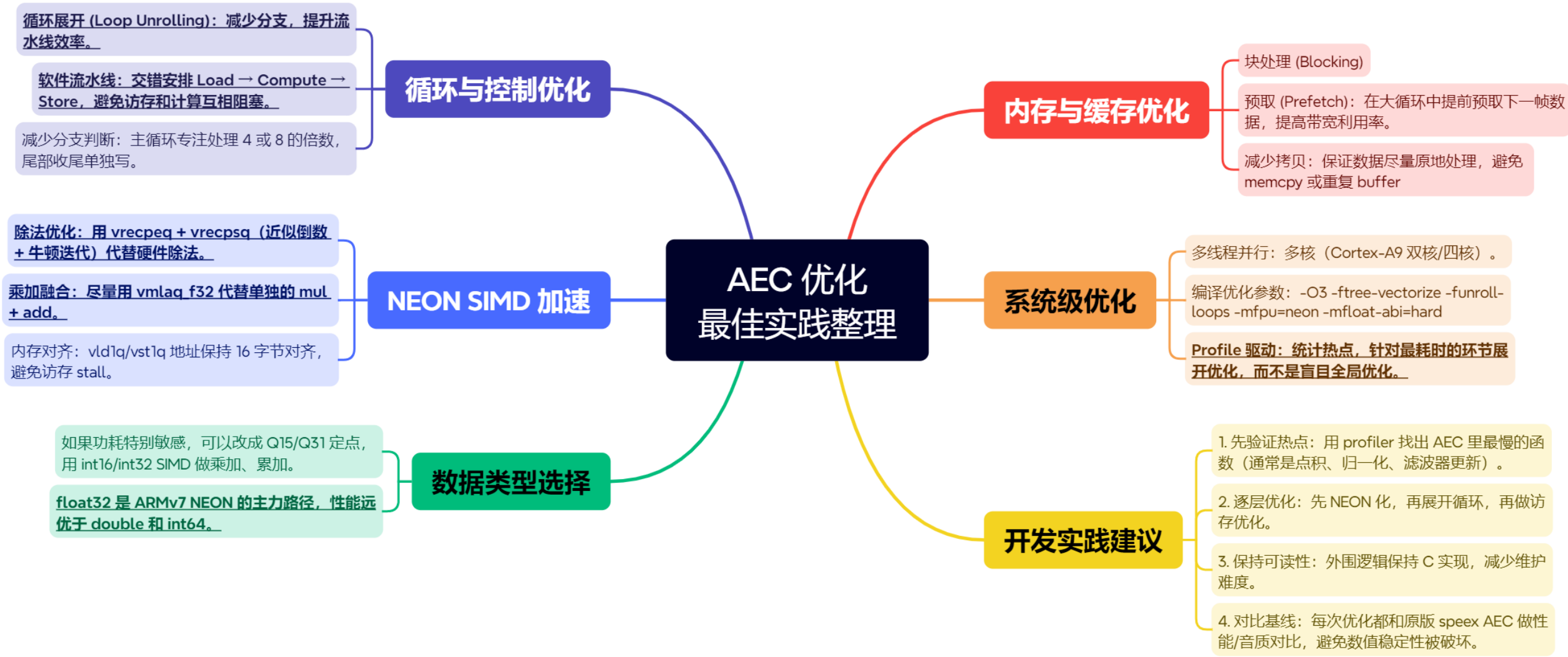
NEON 是 ARM 的 SIMD（单指令处理多个数据）向量运算单元，能够在**结果不变**的前提下**缩短执行时间**、降低 CPU 消耗。使用 128 位宽寄存器，可以分割成 4 个 float32 使用。

主要优化原理

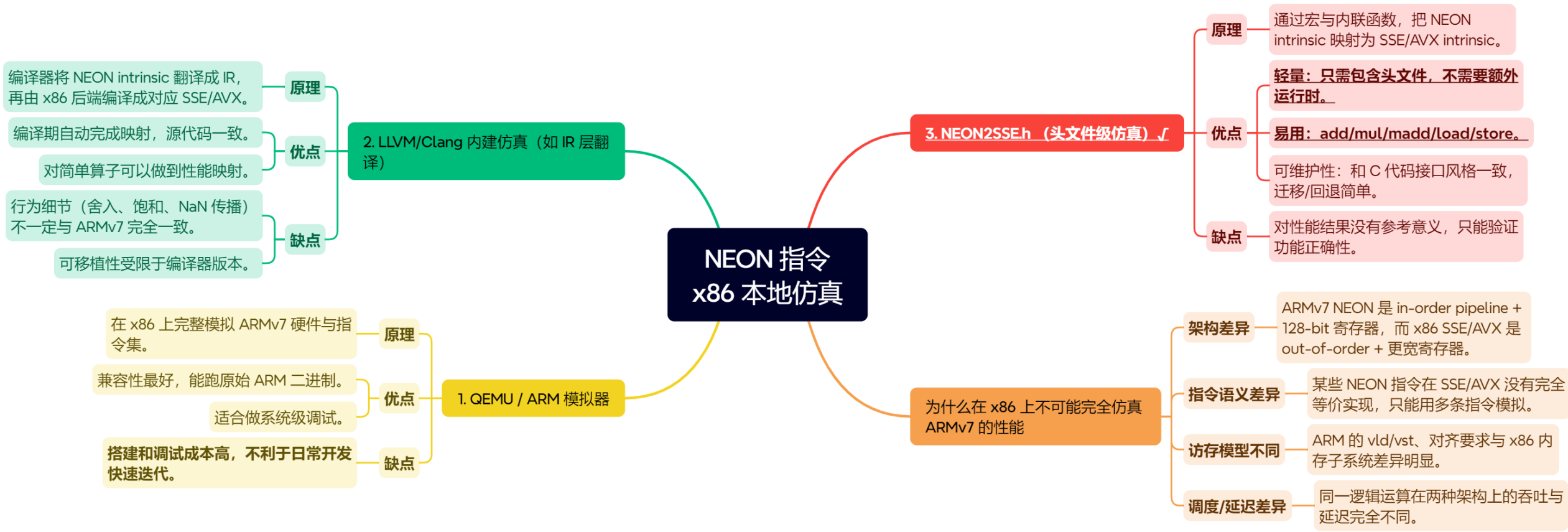
1. 高效的并行计算实现。
2. 硬件级的专门的浮点 SIMD 单元。
3. NEON 专门的高速寄存器。



调研 NEON 的最佳实践



NEON 指令本地仿真环境



本地仿真

对比 ARM 模拟器/虚拟机方案，NEON2SSE.h 仿真方案更为**轻量**、易用。
无需修改源码或调整现有工作流，仅需引入一个头文件，即可在 x86 平台上实现跨平台编译与调试，**环境配置简单、稳定。**

实机 NEON 代码开发

```
for(i = 0; i <= st->frame_size; i++)
{
    float r, e;
    /* Compute frequency-domain adaptation mask */
    r = st->leak_estimate * st->Yf[i];
    e = st->Rf[i] + 1;
    r = MIN32(r, 0.5f * e);
    r = 0.7f * r + 0.3f * (RER * e);
    st->power_1[i] = r /  (e * (st->power[i] + FLT_MIN));
    st->tmp_array_1[i] = st->power_1[i];
}}
```

```
for (; i + 4 <= N; i += 4) {
    float32x4_t vY = vld1q_f32(&Yf[i]);
    float32x4_t vR = vld1q_f32(&Rf[i]);
    float32x4_t vP = vld1q_f32(&power[i]);

    float32x4_t tmp  = vmulq_f32(vY, v_leak);          // Yf * leak
    float32x4_t tmp1 = vaddq_f32(vR, v1);              // Rf + 1
    float32x4_t tmp2 = vmulq_f32(tmp1, v_half);        // tmp1 * 0.5
    tmp = vminq_f32(tmp, tmp2);                        // min(tmp, tmp2)
    tmp2 = vmulq_f32(tmp1, v_rer);                     // tmp1 * RER
    tmp  = vmlaq_f32(vmulq_f32(tmp, v07), tmp2, v03);  // tmp*0.7 + tmp2*0.3
    tmp2 = vaddq_f32(vP, vfltmin);                     // power + FLT_MIN
    tmp1 = vmulq_f32(tmp1, tmp2);                      // tmp1 * tmp2
    float32x4_t result = vdivq_f32_my(tmp, tmp1);         // tmp / tmp1

    vst1q_f32(&power_1[i], result);
}
```

代码NEON优化（部分）

```
Mem: 46016K used, 19340K free, 0K shrd, 3198069688K buff, 377688K cached
CPU:  33% usr  21% sys   0% nic  44% idle   0% io   0% irq   1% sirq
Load average: 18.10 9.51 3.83 1/98 2725
  PID  PPID USER     STAT   VSZ %VSZ %CPU COMMAND
  890     1 root     S     109m 171%  11% {ai_recv} /bin/main
  915     1 root     S     109m 171%   9% {ams} /bin/main
  822     1 root     S     109m 171%   9% {3A_Proc_0} /bin/main
  899     1 root     S     109m 171%   6% {risp} /bin/main
 1122     1 root     S     109m 171%   2% {inet_loop} /bin/main
  922     1 root     D     109m 171%   1% {dla_thread} /bin/main
 1037     1 root     S     109m 171%   1% {inet_loop} /bin/main
  921     1 root     S     109m 171%   1% {algo_recv} /bin/main
  887     1 root     S     109m 171%   1% {video_recv0} /bin/main
  897     1 root     S     109m 171%   1% {playback} /bin/main
  715     1 root     S     109m 171%   1% {inet_loop} /bin/main
  888     1 root     S     109m 171%   1% {video_recv1} /bin/main
 1117     1 root     S     109m 171%   1% {dla_recv} /bin/main
 1016     1 root     S     109m 171%   1% {inet_loop} /bin/main
  842     1 root     S     109m 171%   1% {audio_play} /bin/main
  929     1 root     S     109m 171%   1% {ptz_handle} /bin/main
  889     1 root     S     109m 171%   0% {video_recv3} /bin/main
  854     1 root     S     109m 171%   0% {inet_loop} /bin/main
  894     1 root     S     109m 171%   0% {video_recv10} /bin/main
```

```
Mem: 46224K used, 19132K free, 0K shrd, 3199105976K buff, 376896K cached
CPU:  34% usr  15% sys   0% nic  50% idle   0% io   0% irq   0% sirq
Load average: 18.39 10.40 4.31 2/98 3003
  PID  PPID USER     STAT   VSZ %VSZ %CPU COMMAND
  915     1 root     R     109m 171%   9% {ams} /bin/main
  822     1 root     S     109m 171%   8% {3A_Proc_0} /bin/main
  890     1 root     R     109m 171%   7% {ai_recv} /bin/main
  899     1 root     S     109m 171%   5% {risp} /bin/main
  821     2 root     DW       0   0%   3% [isp0_P0_MAIN]
  922     1 root     S     109m 171%   2% {dla_thread} /bin/main
 2847   684 root     R      956   1%   2% top
  855     2 root     DW       0   0%   2% [venc0_P0_MAIN]
  752     2 root     DW       0   0%   2% [IspDriverThread]
  715     1 root     S     109m 171%   1% {inet_loop} /bin/main
  854     1 root     S     109m 171%   1% {inet_loop} /bin/main
  894     1 root     S     109m 171%   1% {video_recv10} /bin/main
 1122     1 root     S     109m 171%   1% {inet_loop} /bin/main
 1037     1 root     S     109m 171%   1% {inet_loop} /bin/main
  921     1 root     S     109m 171%   1% {algo_recv} /bin/main
  885     2 root     DW       0   0%   1% [ai0_P0_MAIN]
  250     2 root     IW       0   0%   1% [kworker/0:2-eve]
  887     1 root     S     109m 171%   0% {video_recv0} /bin/main
  889     1 root     S     109m 171%   0% {video_recv3} /bin/main
```

C225v2 双讲优化 CPU 占用（top命令）

课题完成进度 - 整体 60%

一、调研 NEON 的最佳实践 - Done

- 选用 Ne10-FFT 替换已有 FFT
- 选用 Intrinsics 实现热点代码
- NEON 汇编实现难以维护，不考虑

二、NEON 指令本地仿真环境 - Done

- 编译命令添加 -msse4.1 -DUSE_SSE4
- 添加 NEON_2_SSE.h 支持本地仿真，模拟 NEON 内在函数的行为

三、代码热点摸排 - 50%

- 目前仅摸排 mdf.c 部分函数 - 50%
- 先验选择 FFT、向量内积等热点代码块进行优化 - 80%

四、实机 NEON 代码开发 - 50%

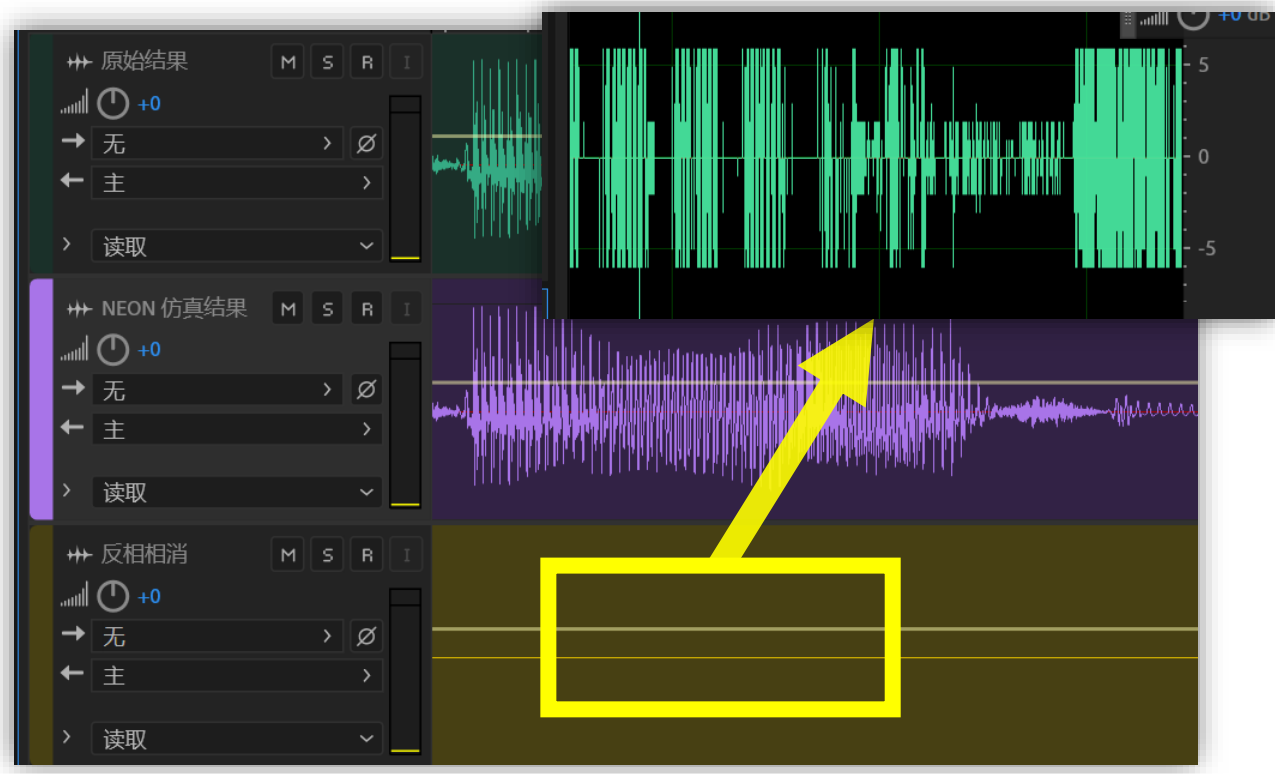
- RTF 模块开发进度 - 90%
- Ne10-FFT 开发进度 - Done
- 回声消除优化开发进度 - 50%

五、实机测试运行 - 30%

- C225v2(ARMv7)测试上述代码，dump结果一致，双讲cpu 占用降低20%

```
[AMS] [RERMY_VECTORIZE] total=142 ms, RTF=0.139 over 32 calls, Amplitude=117.36%
[AMS] [RERNO_VECTORIZE] total=140 ms, RTF=0.137 over 32 calls, Amplitude=89.17%
[AMS] [RERMY_VECTORIZE] total=141 ms, RTF=0.138 over 32 calls, Amplitude=99.30%
[AMS] [RERNO_VECTORIZE] total=157 ms, RTF=0.153 over 32 calls, Amplitude=112.14%
[AMS] [RERMY_VECTORIZE] total=124 ms, RTF=0.121 over 32 calls, Amplitude=87.94%
[AMS] [RERNO_VECTORIZE] total=142 ms, RTF=0.139 over 32 calls, Amplitude=90.45%
[AMS] [RERMY_VECTORIZE] total=114 ms, RTF=0.111 over 32 calls, Amplitude=91.94%
[AMS] [RERNO_VECTORIZE] total=154 ms, RTF=0.150 over 32 calls, Amplitude=108.45%
[AMS] [RERMY_VECTORIZE] total=121 ms, RTF=0.118 over 32 calls, Amplitude=106.14%
[AMS] [RERNO_VECTORIZE] total=138 ms, RTF=0.135 over 32 calls, Amplitude=89.61%
[AMS] [RERMY_VECTORIZE] total=154 ms, RTF=0.150 over 32 calls, Amplitude=127.27%
[AMS] [RERNO_VECTORIZE] total=145 ms, RTF=0.142 over 32 calls, Amplitude=105.07%
[AMS] [RERMY_VECTORIZE] total=136 ms, RTF=0.133 over 32 calls, Amplitude=88.31%
[AMS] [RERNO_VECTORIZE] total=151 ms, RTF=0.147 over 32 calls, Amplitude=104.14%
[AMS] [RERMY_VECTORIZE] total=134 ms, RTF=0.131 over 32 calls, Amplitude=98.53%
[AMS] [RERNO_VECTORIZE] total=107 ms, RTF=0.104 over 32 calls, Amplitude=70.86%
[AMS] [RERMY_VECTORIZE] total=143 ms, RTF=0.140 over 32 calls, Amplitude=106.72%
[AMS] [RERNO_VECTORIZE] total=148 ms, RTF=0.145 over 32 calls, Amplitude=138.32%
[AMS] [RERMY_VECTORIZE] total=168 ms, RTF=0.164 over 32 calls, Amplitude=117.48%
[AMS] [RERNO_VECTORIZE] total=144 ms, RTF=0.141 over 32 calls, Amplitude=97.30%
[AMS] [RERMY_VECTORIZE] total=122 ms, RTF=0.119 over 32 calls, Amplitude=72.62%
```

RTF 模块计时，对比不同的 SIMD 实现



采样点反相相消结果，计算误差小于浮点运算误差

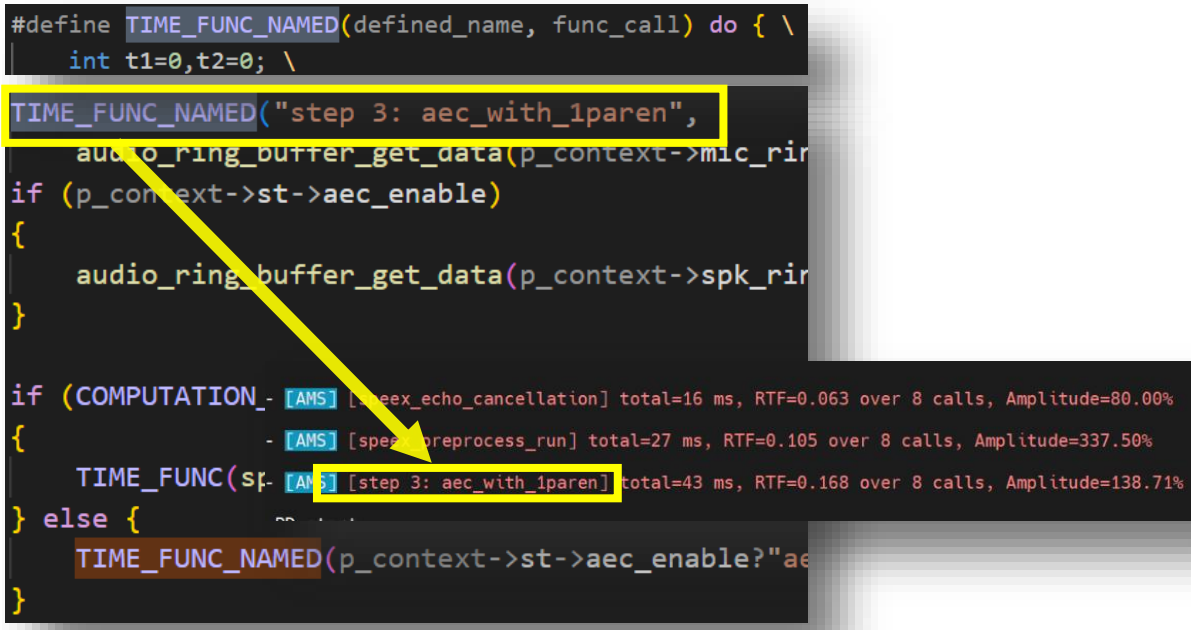
创新点

音频NEON

1. 音频处理中的复杂度非常集中，主要集中在 FFT、自适应滤波、向量内积，且运算非常对称，并行条件好。
2. 实机验证了多个 for 的 NEON 实现效果较差。分析原因是**循环分支开销**成为瓶颈。
3. 在C225v2实机得到了一组明确的算子优化系数，如右下图。由表可以近似计算各代码块 NEON 优化的预期收益。（统计其中各算子占比，并对优化系数加权求和）
4. 据此分析了**浮点转定点运算**明确无法带来优化。即使考虑逆元优化除法。

基于宏定义的 rtf 模块实现

支持多feature，包括代码块/函数计时，隔点计时、多点平均、平滑、变化幅度统计、高精度时间、自定义配置。



RTF 快速应用到代码块

c225v2	NEON	C实现耗时	优化系数	简介
vld1q_f32	248	98.5	**2.517↑**	加载到向量
vmulq_f32	154	254.5	0.605 ↓	向量乘
vaddq_f32	165	275	0.601 ↓	向量加
vminq_f32	179.5	243	0.738 ↓	向量取MIN
vmlaq_f32	164	328.5	**0.499↓**	向量乘加
vst1q_f32	238	154	**1.545↑**	从向量保存
vdupq_n_f32	223	273	0.816 ↓	从标量复制

NEON 各命令优化系数表（NEON耗时÷C实现耗时）

后续工作方向

观察到	优化思路
vdiv 命令在 armv7 不支持	vrecpeq 求倒数+牛顿迭代法
for 循环分支开销大 vld 和 vst 是算子性能瓶颈	循环展开+软件流水线 提高硬件单元的吞吐率
算子耗时 vmla ＝vmul < vmul+vadd	NEON 乘加融合
目前仅优化回声消除	扩大范围摸排代码
vld 和 vst 耗时长	优化 AEC 代码结构 减少中间变量的 vld 和 vst

THANKS

tp-link